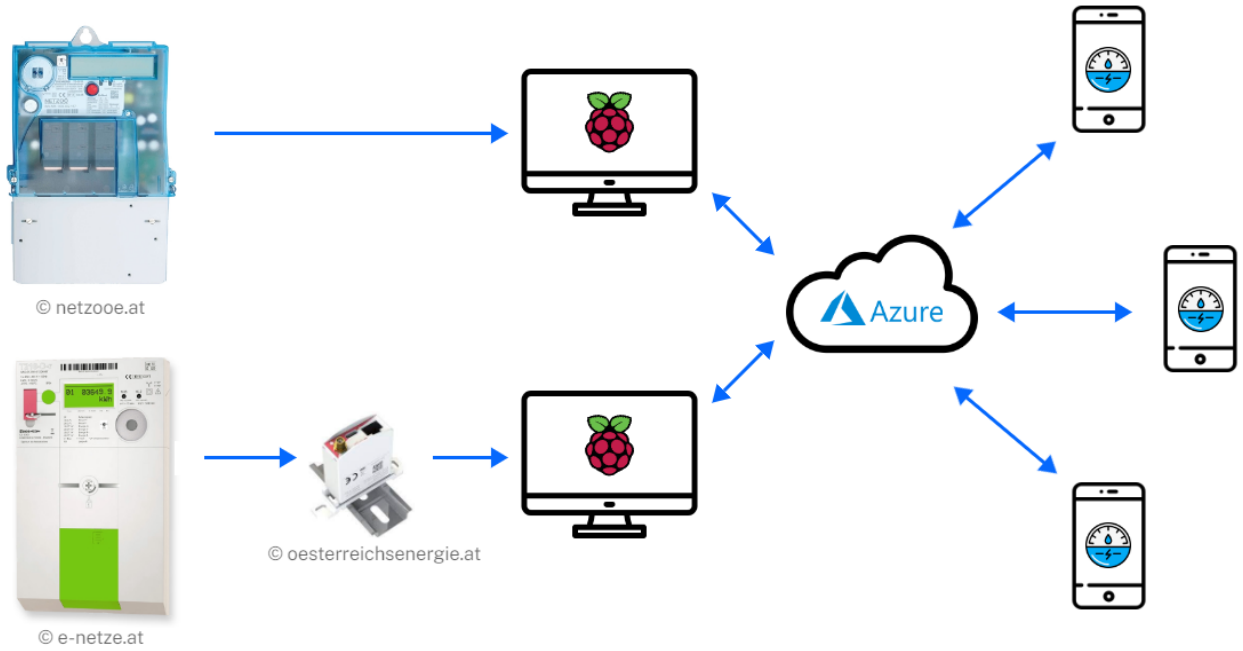
Architektur

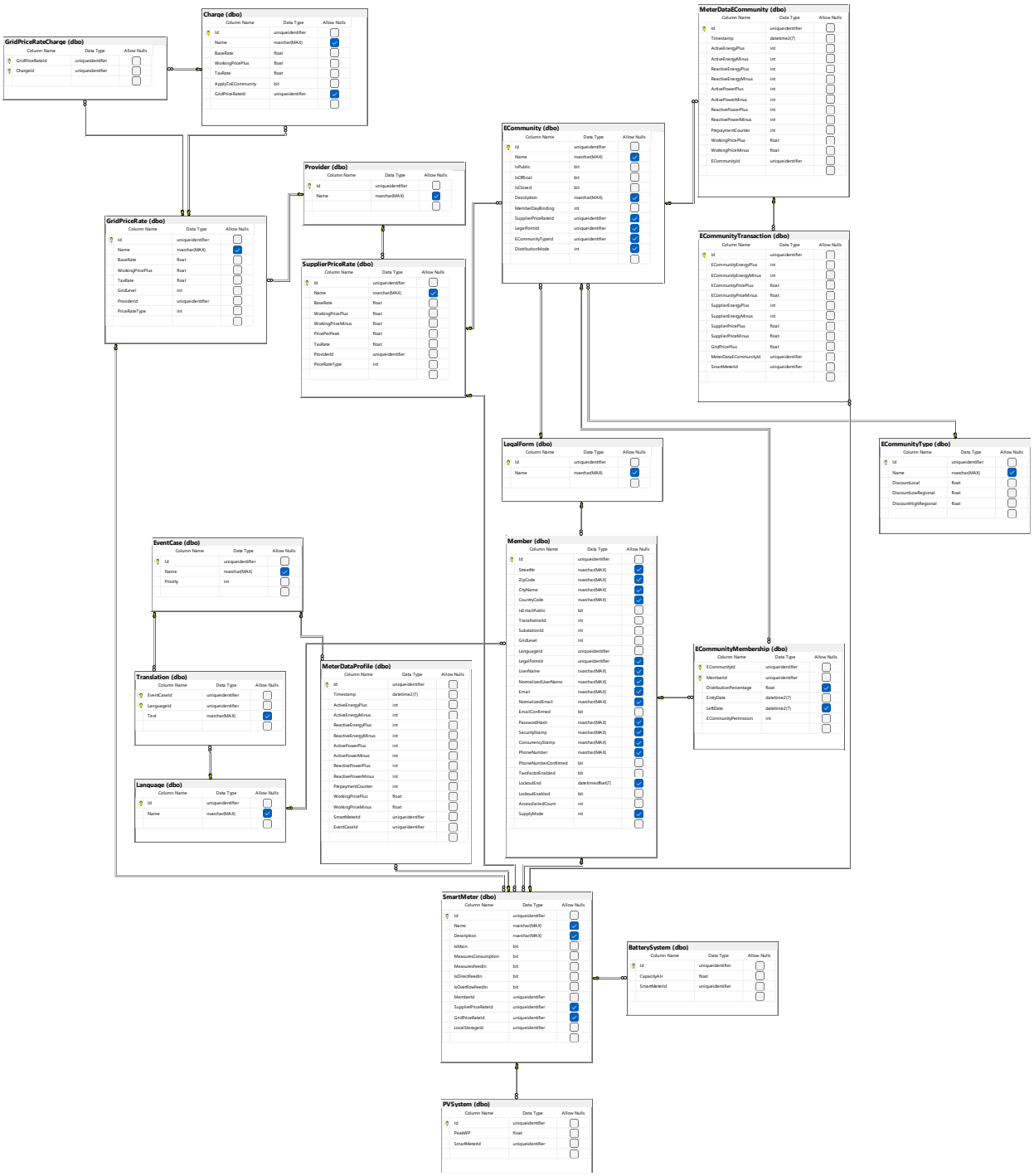
- Smart Meter
  - digitales Messgerät für den Energieverbrauch/-erzeugung
  - Schnittstelle zwischen Haushalt und Netzbetreiber
    - weiters auch zwischen Haushalt und unserer Plattform
  - Kundenschnittstelle **AMIS Zähler** der Energie AG
    - Infrarotschnittstelle
    - liefert jede Sekunde einen Messwert
    - Verschlüsselung der Daten mittels AES-Key
    - künftig einheitliche Schnittstelle in Österreich
- Raspberry
  - liest die Energiedaten mittels der Kundenschnittstelle vom Smart Meter aus
    - C++ Service (programmiert von Energie AG und abgeändert durch uns)
      - persistiert historische Daten alle 15 min
      - speichert Sekundendaten für die letzten 90 Sekunden
  - .NET Core Backend
    - bearbeitet die Energiedaten entsprechend für die Plattform
    - falls Anomalien auftreten (Abweichung vom definierten Profil)
      - Raspberry schickt Daten an Cloud
    - Datenaustausch von/zur Cloud mittels SignalR
      - vor allem Echtzeitdaten
    - Datenaustausch zur Cloud teilweise mit REST (ohne Echtzeitrelevanz)
  - SQLite3 Datenbank
- Cloud (Azure)
  - .NET Core Backend
    - Handler für alle Energiegemeinschaften und deren Mitglieder inklusive Smart Meter und Tarifierung
      - speichert Profil von jedem Smart Meter
  - MS-SQL Datenbank
- Endgeräte (Android)
  - Schnittstelle zum Mitglied
  - Echtzeitdaten werden mittels SignalR empfangen
  - andere Daten werden über REST-Requests abgefragt


© netzooe.at
© e-netze.at
© oesterreichsenergie.at
Azure

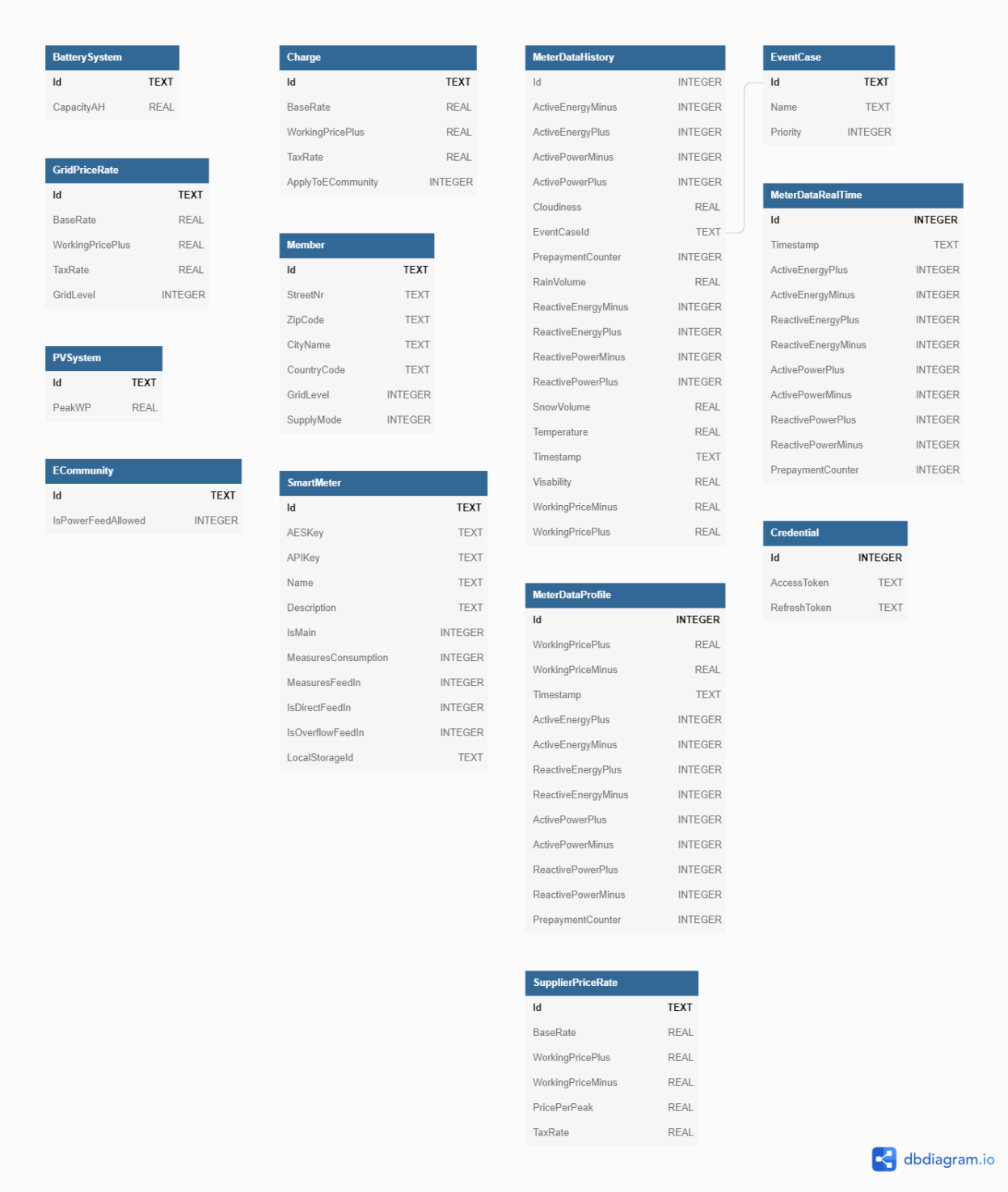
Datenmodelle

Beschreibung der Attribute kann man sich im Source Code ansehen.

Cloud (Azure)



Lokal (Raspberry)



App (Android)

```
@Entity(tableName = "tile")
data class Tile(
    @PrimaryKey
    var tileId: String,
    var region: String,
    var action: String,
    var value: String,
    var isVisible: Boolean,
    var isLargeTile: Boolean = false,
    var order: Int,
    var colorId: Int) {}
```

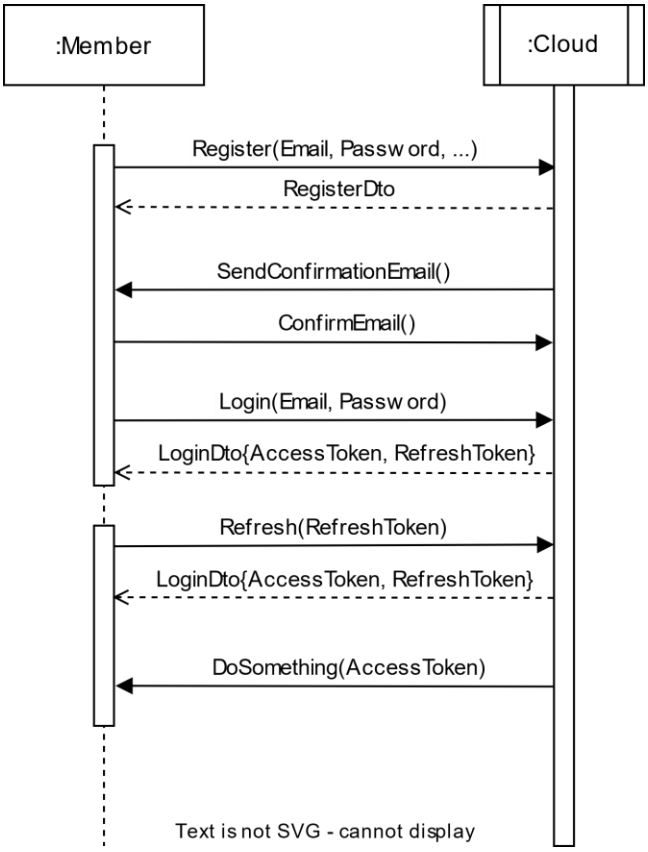
```
@Entity(tableName = "smart_meter")
data class SmartMeter(
    @PrimaryKey var ID: String,
    var name: String,
    var description: String
)
```

```
@Entity (tableName = "member")
data class Member(
    @PrimaryKey var memberId: String,
    var username: String,
    var email: String,
    var password: String,
    var accessToken: String,
    var refreshToken: String,
    var languageName: String)
```

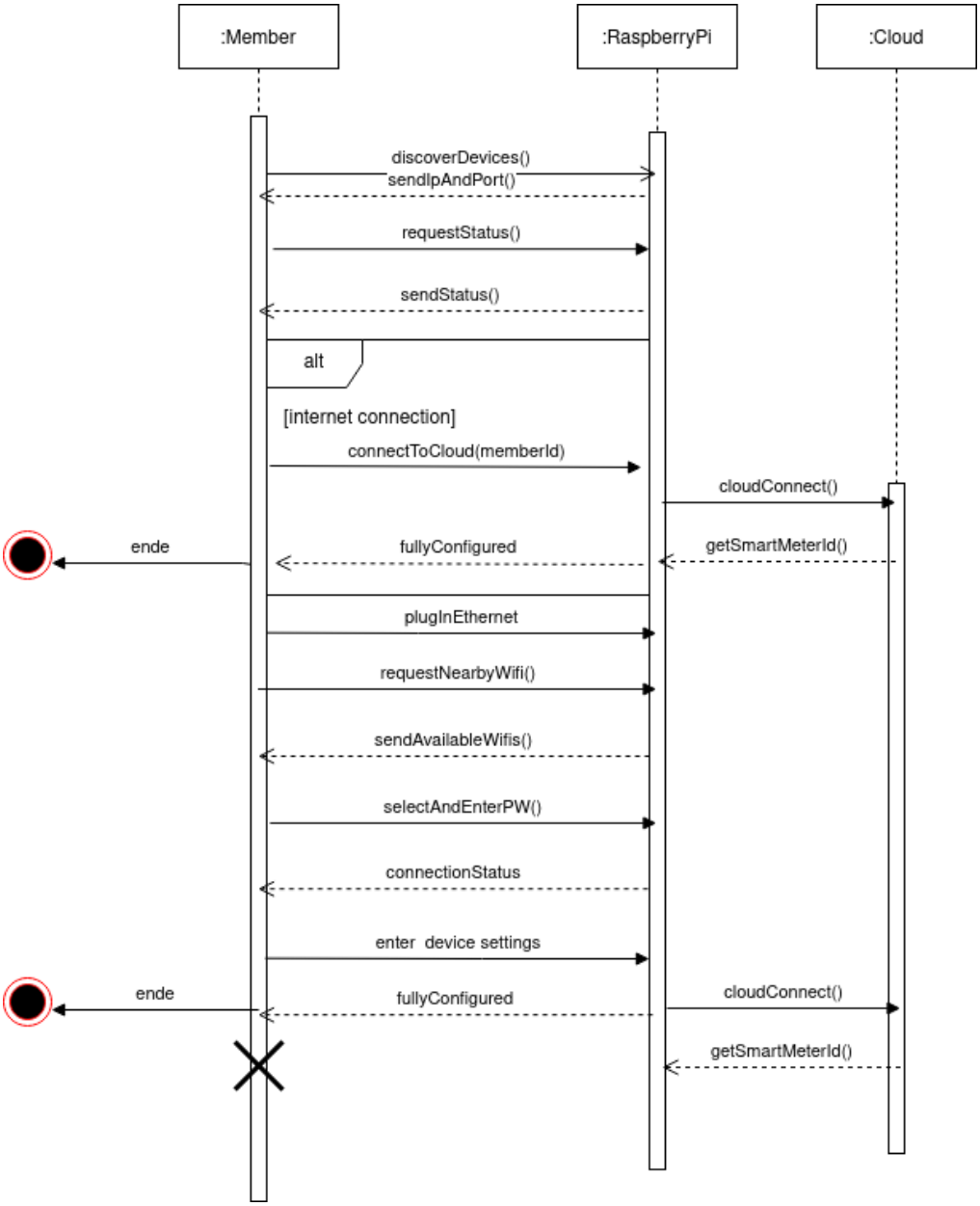
Abläufe

Authentifizierung

- Login
  - Member muss E-Mail bestätigt haben
  - E-Mail und Passwort müssen natürlich übereinstimmen
  - bei 3 fehlgeschlagenen Versuchen
    - Member für 5 Minuten gesperrt
- Access Token
  - gültig für 10 Minuten
  - Zugriff auf Ressourcen (REST und SignalR)
- Refresh Token
  - gültig für 90 Tage
  - man erhält neuen Access und Refresh Token



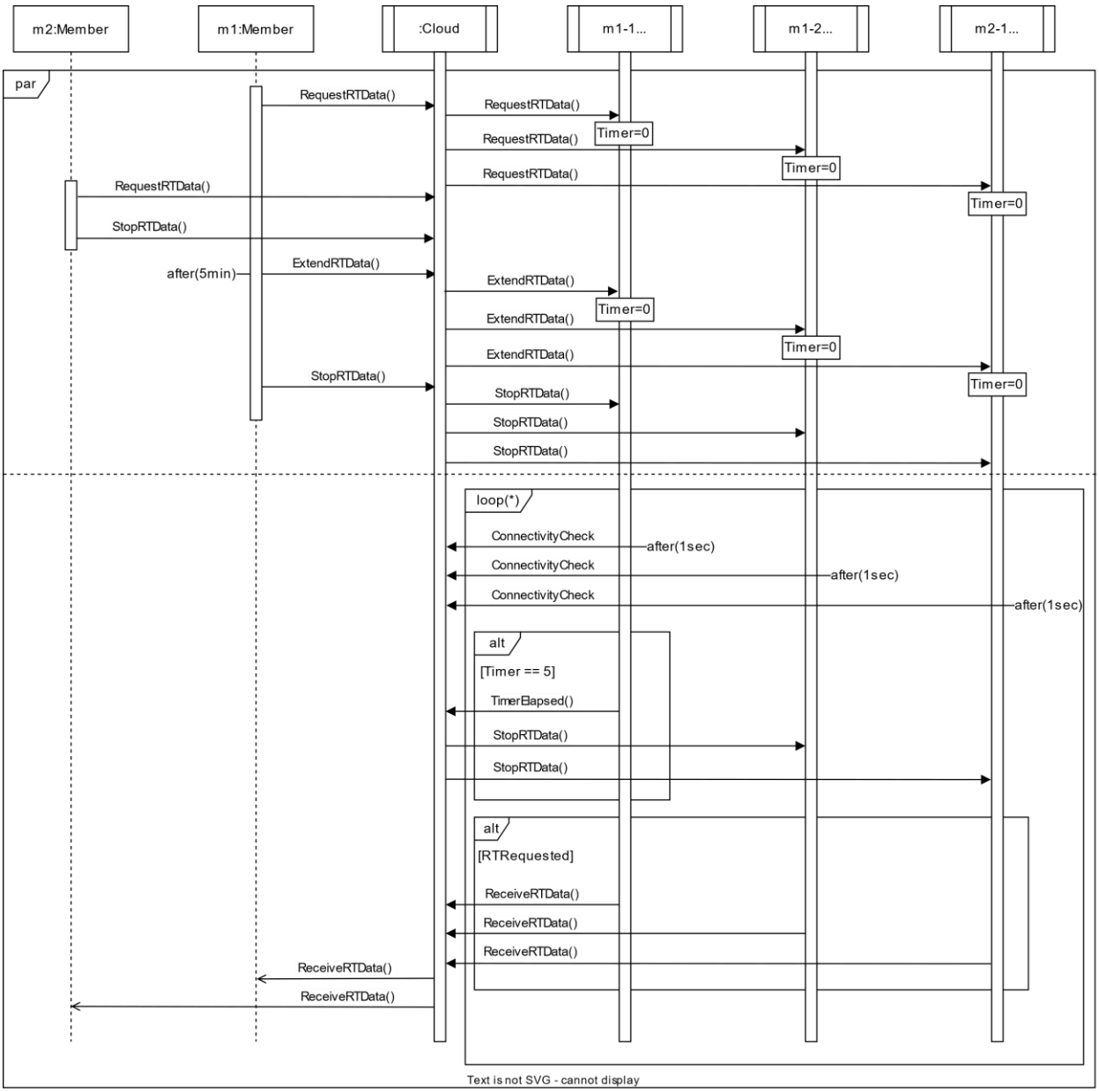
Pairing



Real Time

- Abwicklung des Echtzeit-Traffics mittels SignalR
- `RequestRTData`
  - Endgerät-Session starten
  - Raspberry-Sessions starten
    - falls nicht bereits gestartet
    - bei ersten Gerät von Energiegemeinschaft
- `ExtendRTData`
  - Endgeräte müssen alle 5 Minuten ihre Session verlängern

            - Verhinderung von unnötigen Traffic
- `StopRTData`
    - Endgerät-Session stoppen
    - Raspberry-Sessions stoppen
        - falls alle Endgerät-Sessions geschlossen
- jede Sekunde durchläuft ein Raspberry die Schleife
- `ConnectivityCheck`
    - überprüft ob SignalR-Verbindung noch offen ist
    - falls geschlossen: neu verbinden
- `TimerElapsed`
    - Timer ist abelaufen
    - Echtzeit-Session stoppen
    - `ExtendRTData` wurde nicht aufgerufen
- `ReceiveRTData`
    - aktueller Messwert an Cloud senden


m2:Member
m1:Member
:Cloud
m1-1...
m1-2...
m2-1...
par
RequestRTData()
RequestRTData()
RequestRTData()
Timer=0
RequestRTData()
Timer=0
RequestRTData()
Timer=0
RequestRTData()
StopRTData()
after(5min)
ExtendRTData()
ExtendRTData()
Timer=0
ExtendRTData()
Timer=0
ExtendRTData()
Timer=0
StopRTData()
StopRTData()
StopRTData()
StopRTData()
loop(*)
ConnectivityCheck
after(1sec)
ConnectivityCheck
after(1sec)
ConnectivityCheck
after(1sec)
alt
[Timer == 5]
TimerElapsed()
StopRTData()
StopRTData()
alt
[RTRequested]
ReceiveRTData()
ReceiveRTData()
ReceiveRTData()
ReceiveRTData()
ReceiveRTData()
Text is not SVG - cannot display